# The problem in one slide

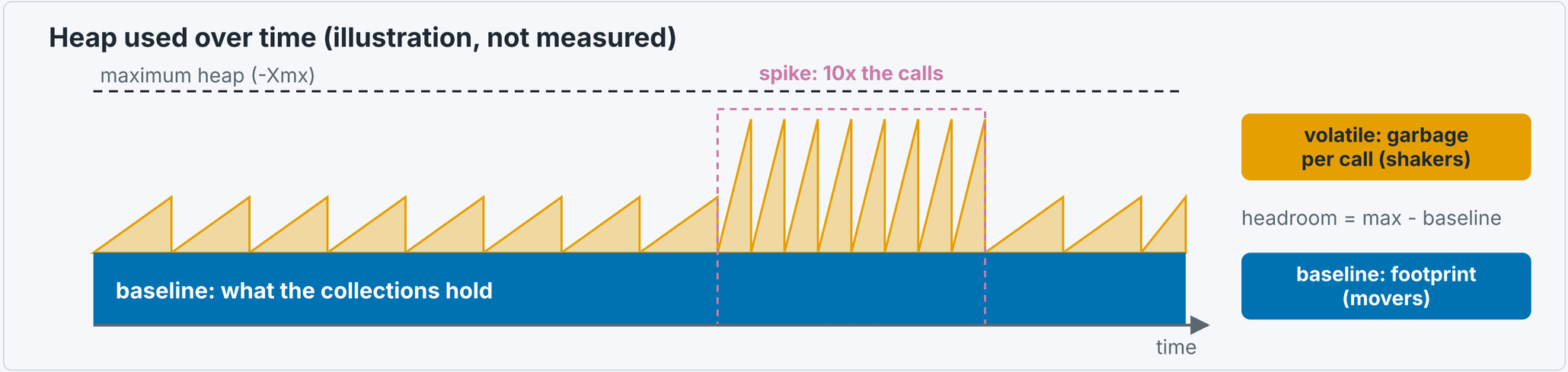


JDK collections and streams spend heap on objects the code never mentions.

- Boxed `Integers` and per-entry nodes inflate what a collection holds.
- Every stream call builds a pipeline of objects and drops it.
- How much garbage depends on the JIT, the `Integer` cache and the headers.
- Fat-free (non-capturing) lambdas and plain loops leave far less behind.

# Baseline, churn and a spike






- Headroom is what is left above the baseline for garbage.
- A spike multiplies the garbage rate; the baseline stays where it was.

# Why the volatile part is the risk



Stream garbage grows with calls per second, not with the number of elements.

- One JDK `anyMatch` call leaves 176 B, wherever the match is.
- At 10 M calls/s that is 1.76 GB/s of young-generation garbage.
- A 10 s spike at that rate allocates about 17.6 GB.
- The same question as an Eclipse Collections loop allocates 0 B.